

Secure Multiplayer Lobby Protocol

Autor: Konrad Iwanczewski

Github: <https://github.com/AtelierRq/Secure-Multiplayer-Lobby-Protocol>

Etap 2 — Projekt aplikacji wykorzystującej protokół

1. Opis funkcjonalny aplikacji

a) Co aplikacja robi?

Projektowana aplikacja jest systemem klient–serwer wykorzystującym autorski protokół SMLP (Secure Multiplayer Lobby Protocol).

Aplikacja umożliwia użytkownikom:

- łączenie się z serwerem multiplayer,
- logowanie przy użyciu nickname,
- tworzenie lobby,
- dołączanie do lobby,
- komunikację tekstową pomiędzy uczestnikami,
- zmianę statusu gotowości,
- rozpoczęcie sesji multiplayer,
- zarządzanie lobby przez hosta.

Aplikacja pełni rolę warstwy organizującej sesję multiplayer i komunikację pomiędzy graczami.

b) Jaki problem użytkownika rozwiązuje?

Aplikacja rozwiązuje problem organizacji i synchronizacji komunikacji pomiędzy wieloma użytkownikami uczestniczącymi w sesji multiplayer.

System umożliwia:

- tworzenie wspólnych lobby,
- zarządzanie uczestnikami,
- synchronizację gotowości graczy,
- bezpieczną komunikację,
- rozpoczęcie wspólnej sesji.

c) Kto jest użytkownikiem (aktorzy)?

PLAYER

Standardowy użytkownik aplikacji.

Może:

- logować się,
- dołączać do lobby,
- wysyłać wiadomości,
- oznaczać gotowość,
- opuszczać lobby.

HOST

Użytkownik będący właścicielem lobby.

Dodatkowo może:

- rozpocząć sesję multiplayer,
- usuwać lobby,
- zarządzać uczestnikami lobby.

SERVER

Centralny komponent systemu.

Odpowiada za:

- utrzymywanie połączeń,

- zarządzanie lobby,
- synchronizację stanu użytkowników,
- autoryzację,
- bezpieczeństwo komunikacji.

2. Architektura rozwiązania

a) Model komunikacji

Aplikacja działa w modelu klient–serwer.

Klient <----TLS/TCP----> Serwer

Serwer jest centralnym punktem komunikacji.

Klienci nie komunikują się bezpośrednio pomiędzy sobą.

b) Komponenty systemu

1. Klient CLI

Komponent odpowiedzialny za:

- połączenie z serwerem,
- wysyłanie komunikatów SMLP,
- odbieranie odpowiedzi,
- wyświetlanie informacji użytkownikowi.

Klient będzie działał w trybie terminalowym (CLI).

2. Serwer SMLP

Główny komponent systemu.

Odpowiada za:

- obsługę wielu klientów,
- zarządzanie sesjami,
- zarządzanie lobby,
- walidację komunikatów,
- autoryzację użytkowników,
- synchronizację stanu.

3. Moduł bezpieczeństwa

Odpowiada za:

- TLS,
- walidację certyfikatów,
- zarządzanie tokenami sesyjnymi,
- ochronę przed nieautoryzowanym dostępem.

4. Moduł lobby

Odpowiada za:

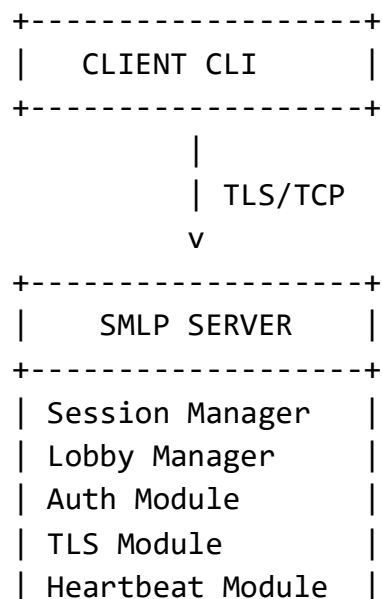
- tworzenie lobby,
- listę uczestników,
- role HOST/PLAYER,
- synchronizację statusu READY.

5. Moduł heartbeat

Odpowiada za:

- wykrywanie timeoutów,
- PING/PONG,
- usuwanie nieaktywnych klientów.

c) Diagram komponentów



+-----+

d) Przepływ danych pomiędzy komponentami

1. Klient wysyła komunikat SMLP.
2. Serwer odbiera komunikat.
3. Serwer waliduje dane.
4. Serwer aktualizuje stan lobby.
5. Serwer rozsyła aktualizacje do klientów.
6. Klienci odbierają komunikaty i aktualizują stan lokalny.

3. Przypadki użycia (Use Cases)

Use Case 1 — Logowanie użytkownika

Cel

Uzyskanie dostępu do systemu multiplayer.

Aktor

PLAYER

Warunki wstępne

- aktywne połączenie TLS,
- nickname nie jest zajęty.

Scenariusz główny

1. Klient wysyła LOGIN.
2. Serwer sprawdza nickname.
3. Serwer tworzy sesję.
4. Serwer zwraca token sesyjny.

Scenariusze alternatywne / błędy

- nickname zajęty,
- błędny format komunikatu,
- timeout połączenia.

Wynik końcowy

Użytkownik zostaje zalogowany.

Use Case 2 — Utworzenie lobby

Cel

Utworzenie nowej sesji multiplayer.

Aktor

HOST

Warunki wstępne

- użytkownik zalogowany.

Scenariusz główny

1. Użytkownik wysyła CREATE_LOBBY.
2. Serwer tworzy lobby.
3. Użytkownik otrzymuje rolę HOST.
4. Serwer zwraca identyfikator lobby.

Scenariusze alternatywne / błędy

- przekroczony limit lobby,
- błędny format komunikatu.

Wynik końcowy

Lobby zostaje utworzone.

Use Case 3 — Dołączenie do lobby

Cel

Dołączenie do istniejącej sesji multiplayer.

Aktor

PLAYER

Warunki wstępne

- użytkownik zalogowany,
- lobby istnieje,
- lobby nie jest pełne.

Scenariusz główny

1. Klient wysyła JOIN.
2. Serwer dodaje użytkownika do lobby.

3. Serwer wysyła JOIN_OK.
4. Pozostali gracze otrzymują PLAYER_JOINED.

Scenariusze alternatywne / błędy

- lobby nie istnieje,
- lobby pełne,
- użytkownik już znajduje się w lobby.

Wynik końcowy

Użytkownik dołącza do lobby.

Use Case 4 — Wysyłanie wiadomości

Cel

Komunikacja tekstowa pomiędzy uczestnikami lobby.

Aktor

PLAYER

Warunki wstępne

- użytkownik znajduje się w lobby.

Scenariusz główny

1. Klient wysyła CHAT.
2. Serwer waliduje wiadomość.
3. Serwer rozsyła CHAT_BROADCAST.

Scenariusze alternatywne / błędy

- zbyt długa wiadomość,
- użytkownik nie znajduje się w lobby.

Wynik końcowy

Wiadomość zostaje dostarczona uczestnikom lobby.

Use Case 5 — Rozpoczęcie sesji multiplayer

Cel

Rozpoczęcie wspólnej sesji przez wszystkich gotowych graczy.

Aktor

HOST

Warunki wstępne

- wszyscy gracze posiadają status READY,
- użytkownik posiada rolę HOST.

Scenariusz główny

1. Host wysyła START.
2. Serwer sprawdza status graczy.
3. Serwer zmienia stan lobby na IN_GAME.
4. Serwer wysyła GAME_START.

Scenariusze alternatywne / błędy

- nie wszyscy gracze są READY,
- użytkownik nie jest HOSTEM.

Wynik końcowy

Sesja multiplayer zostaje rozpoczęta.

Use Case 6 — Opuszczenie lobby i wylogowanie

Cel

Zakończenie sesji użytkownika.

Aktor

PLAYER / HOST

Warunki wstępne

- użytkownik połączony z serwerem.

Scenariusz główny

1. Użytkownik wysyła LEAVE.
2. Serwer usuwa użytkownika z lobby.
3. Jeśli lobby jest puste — zostaje usunięte.
4. Użytkownik wysyła LOGOUT.
5. Serwer kończy sesję.

Scenariusze alternatywne / błędy

- utrata połączenia,
- timeout klienta.

Wynik końcowy

Użytkownik zostaje wylogowany.

4. Mapowanie aplikacji na protokół

a) Funkcje aplikacji i komunikaty protokołu

FUNKCJA APLIKACJI	KOMUNIKATY SMLP
LOGOWANIE	LOGIN, LOGIN_OK
TWORZENIE LOBBY	CREATE_LOBBY, LOBBY_CREATED
DOŁĄCZENIE DO LOBBY	JOIN, JOIN_OK
KOMUNIKACJA TEKSTOWA	CHAT, CHAT_BROADCAST
SYNCHRONIZACJA GRACZY	READY, NOT_READY
ROZPOCZĘCIE SESJI	START, GAME_START
HEARTBEAT	PING, PONG
OBSŁUGA BŁĘDÓW	ERROR
ROZŁĄCZENIE	LEAVE, LOGOUT, BYE

b) Wsparcie przypadków użycia przez protokół

SMLP zapewnia:

- zarządzanie stanami klientów,
- synchronizację lobby,
- bezpieczną komunikację,
- obsługę timeoutów,
- obsługę błędów,
- zarządzanie sesją multiplayer.

c) Możliwe rozszerzenia protokołu

Możliwe przyszłe rozszerzenia:

- matchmaking,
- ranking graczy,
- prywatne lobby,
- lista znajomych,
- reconnect do sesji,
- synchronizacja właściwej gry.

5. Wymagania niefunkcjonalne

a) Bezpieczeństwo

System wykorzystuje:

- TLS 1.2,
- certyfikaty SSL,
- tokeny sesyjne,
- walidację komunikatów,
- ograniczenia długości danych.

b) Wydajność

System powinien obsługiwać:

- wielu klientów jednocześnie,
- wiele lobby,
- komunikację w czasie rzeczywistym.

Maksymalne opóźnienie odpowiedzi powinno być możliwie niskie.

c) Niezawodność

System powinien:

- wykrywać timeouty,
- obsługiwać utratę połączenia,
- poprawnie usuwać nieaktywnych klientów,
- utrzymywać spójność stanu lobby.

d) Skalowalność

Architektura powinna umożliwiać:

- zwiększenie liczby klientów,
- zwiększenie liczby lobby,
- dalszy rozwój funkcjonalności.

e) Logowanie i diagnostyka

Serwer powinien prowadzić logi:

- połączeń,
- błędów,
- timeoutów,
- działań użytkowników,
- zdarzeń bezpieczeństwa.

6. Plan implementacji i testowania

a) MVP (Minimum Viable Product)

Minimalna wersja aplikacji powinna zawierać:

- klient CLI,
- serwer,
- TLS,
- logowanie nickname,
- tworzenie lobby,
- dołączanie do lobby,
- komunikację tekstową,
- READY / START,
- heartbeat.

b) Plan testów

Testy funkcjonalne

- poprawne logowanie,
- tworzenie lobby,
- dołączanie do lobby,
- komunikacja pomiędzy klientami,
- rozpoczęcie sesji multiplayer.

Testy błędów

- błędny format komunikatu,
- timeout klienta,

- rozłączenie podczas sesji,
- próba dołączenia do nieistniejącego lobby.

Testy bezpieczeństwa

- próba połączenia bez TLS,
- niepoprawny certyfikat,
- nieautoryzowane rozpoczęcie sesji.

Testy obciążeniowe

- wielu klientów jednocześnie,
- wiele lobby,
- intensywna wymiana komunikatów.

c) Kryteria zakończenia projektu

Projekt zostanie uznany za zakończony, gdy:

- wszystkie podstawowe funkcje będą działały poprawnie,
- komunikacja będzie zgodna z SMLP,
- TLS będzie poprawnie skonfigurowany,
- obsługa błędów będzie działała poprawnie,
- aplikacja przejdzie podstawowe testy funkcjonalne.